Agentic AI Assignment: Prompting Methods and a Calculator Agent on GSM8K

İbrahim Halil Demircioğlu

November 2026

Code: https://github.com/H4LL1L/Agentic-AI-Homework

1. Question 1

All runs were done on a free Colab T4 GPU with at most 512 new tokens per call. The system prompt was "Solve the math problem step by step. End with: The answer is <number>." and the final answer is read from "The answer is X", or from the last sentence if the model does not write it. Self-consistency samples with temperature 0.7. In reflection, the first solution is the four-shot one, and the review call only sees the question and that solution, not the four examples.

Method	Dev (10 questions)	Test accuracy	Generated tokens (test)	Model calls (test)
Zero-shot	5/10	10/30 (33%)	7,681	30
Four-shot	5/10	10/30 (33%)	3,446	30
Self-consistency	5/10	11/30 (37%)	15,784	120
Reflection	8/10	10/30 (33%)	9,687	60

Table 1: Results on test examples 1-30.

Extra inference helped (test question 11)

Downloads were "reduced by 30%" in the third month. Four-shot computed 180 x 0.3 = 54, using the removed part instead of what was left, and answered 394. Self-consistency got 366, 480, 366 and 597, so the vote picked the correct 366. Reflection also fixed the step to 180 x 0.7 = 126 and answered 366.

Extra inference harmed (test question 28)

Four-shot answered correctly: 60 / 15 = 4 cartons, 4 x $4 = $16. In self-consistency two samples made the same mistake (60 / 15 servings per day) and both answered 960, so the wrong answer won the vote. In reflection the first answer was 16, but the review changed it to about $4.27.

Best accuracy-cost tradeoff

Four-shot. It has the same accuracy as zero-shot with less than half of the tokens. Self-consistency was better by only one question, for 4 times the calls and 4.6 times the tokens, and reflection cost almost 3 times the tokens with no gain. With 30 questions, one question of difference is not really meaningful.

2. Question 2

The model writes "CALC: <expression>", my code runs the calculator and sends back "RESULT: <value>" or an error message. The calculator parses the expression with Python's ast module and only evaluates numbers, + - * / and brackets; anything else returns an error. Generation stops when the model starts writing its own RESULT line, so it cannot make up tool results.

Method	Test accuracy	Generated tokens	Model calls	Calculator calls
Self-consistency (strongest Q1 method)	11/30 (37%)	15,784	120	-
Calculator agent	10/30 (33%)	4,028	50	20

Table 2: Calculator agent compared with the strongest Question 1 method.

The agent was one question behind self-consistency but used about a quarter of its tokens. It used the calculator in 19 of the 30 questions. Only one call failed ("5:00 PM - 1:00 PM"); after the error message the model rewrote it as 4 * 2 and answered correctly.

Failure 1: incorrect problem formulation (test question 1)

For Janet's eggs the model wrote CALC: 16 - 3 - 4, the calculator correctly returned 9, and the model answered $9. The arithmetic was right, but the step "x $2" was missing (correct answer 18). 11 of the 20 wrong answers were like this.

Failure 2: arithmetic/copy error (test question 18)

The calculator returned 2275 and the model answered $22,750, so it copied the result wrong. Only 2 of the 20 wrong answers were of this type.

Why successful tool execution does not guarantee correctness

The calculator only checks that the expression is valid and computes it exactly; it cannot know if it is the right expression for the problem. In an earlier version the calculator rejected "16 - 3 - 4 = 7" (question 1) and "4 * 20 + 20 = 100" (question 7), and the model then solved both correctly on its own. When the calculator accepted them, the model took the intermediate result as the final answer and both became wrong.

3. Testing and Improvements

I tested on the development set and read the model's outputs to find problems. The main changes were:

- Token limit from 300 to 512, because some answers were cut off.
- Answer extraction: taking the last number read "earned $10 for 50 minutes" as 50, so I read \boxed{} answers and the final sentence instead.
- Reflection: with the four examples in the review call, the model sometimes checked an example instead of its own answer. Without them, dev went from 5/10 to 8/10 and test from 7/30 to 10/30.
- Self-consistency: the 4 samples are generated in one batched call, which is faster and still counted as 4 calls.
- Agent: clearer calculator error messages, because after a raw Python error the model thought the user made a mistake.

I ran the test set more than once while fixing these. Decisions were made on the development set, except the "=" handling in the calculator, which I added after a test output and which lowered the agent's score by one.

4. What I Learned

I expected bigger differences between the methods. I thought self-consistency and reflection would clearly beat simple prompting, but all four methods got 10 or 11 out of 30, and the extra calls mostly increased the cost. I think the reason is the model size: a 0.5B model often misunderstands a question in the same way, so voting repeats the mistake, and it is not good at checking its own answers.

I also learned that small details like the token limit, the answer extraction and what the model sees in the review call changed the results as much as the methods did. And the calculator never computed anything wrong, but it could not fix wrong reasoning, which caused most of the agent's errors.